

CharacterOS — Epoch 0 Foundation Package

Master Specification Document Version: 0.1.0-DRAFT Status: PROPOSED Owner: Keith Clemente / Feral Gloss Empire Created: 2026-06-26 Ratification Required By: Canon-Lock Authority (Keith Clemente)

Review Protocol: This document is the single source of truth during ratification. All cross-artifact changes should be made here before any artifact is split into a canonical file. Once all four artifacts reach Status: RATIFIED, each section becomes an independent canonical document.

Table of Contents

1. [Artifact 0 — Architectural Laws](#)
 2. [Artifact 1 — Canonical Decisions Register](#)
 3. [Artifact 2 — Universal Module Contract Template](#)
 4. [Artifact 3 — Character Seed Specification v1.0](#)
-
-

Artifact 0 — Architectural Laws

Artifact ID:	FGE-ARCH-LAWS-001
Version:	0.1.0
Status:	PROPOSED
Owner:	Keith Clemente
Last Ratified:	—
Depends On:	Nothing. This is the root artifact.
Required By:	All other CharacterOS artifacts.
Failure Mode:	Laws are too abstract to resolve disputes or too numerous to rem A law that cannot decide a real case is not a law — it is decora
Acceptance Gate:	Every law must answer "yes" to the question: Could a developer w never read any FGE conversation apply this rule unambiguously to module design decision?

Purpose

The Architectural Laws are the immutable constitutional layer of CharacterOS. They exist above all module contracts, specifications, and implementations. When any lower artifact conflicts with a Law, the Law wins. When two Laws appear to conflict, the conflict is a ratification crisis and must be resolved by the Canon-Lock Authority before work continues.

These laws are not guidelines. They do not have exceptions unless a new Law explicitly creates one. They are the rules that govern the rules.

Laws

LAW-001 — Specifications Precede Implementations

No implementation may begin until a ratified specification exists for the module being implemented. A specification is not ratified until it has passed all gates defined in the Universal Module Contract Template.

Rationale: Implementations built without specifications create implicit contracts that are invisible, untestable, and impossible to version. This law exists because the cost of a missing specification is paid by every module that depends on it.

Evidence: Every major drift in FGE governance history has originated from building before defining. The CharacterOS character of the system — governance-first, epoch-driven — requires this constraint to be absolute.

Impact: No module sprint begins without a completed, gate-passed spec. This slows the start; it accelerates everything after.

Revision Policy: Cannot be revised without a versioned replacement Law that addresses the same constraint. The replacement must pass Acceptance Gate before the original is deprecated.

Status: PROPOSED

LAW-002 — Modules Communicate Only Through Contracts

No module may call, read from, write to, or depend on the internal state of another module. All inter-module communication must pass through a defined, versioned contract interface.

Rationale: Modules that reach into each other's internals create invisible coupling. When one changes, the other breaks in ways that are hard to detect and harder to trace. Contracts are the only auditable boundary.

Evidence: This is the standard architectural principle underlying all scalable specification systems (microservices, compiler IR layers, hardware abstraction). The FGE multi-model workflow (Claude → Grok → Canon) already operates on a hand-off contract model; CharacterOS formalizes what is already working.

Impact: Contract definitions must be complete before any two modules can be connected. Increases design time; eliminates a class of integration failures entirely.

Revision Policy: Can be revised to narrow scope (e.g., carve out specific intra-epoch utility patterns) but cannot be broadened to permit undocumented state sharing.

Status: PROPOSED

LAW-003 — Earlier Epochs Cannot Depend on Later Epochs

No artifact, contract, or implementation belonging to Epoch N may reference, require, or assume the existence of any artifact from Epoch N+1 or later.

Rationale: Epoch dependencies that run forward in time destroy the ability to build incrementally. If Epoch I secretly requires something from Epoch III, Epoch I is not actually a foundation — it is a placeholder that cannot be validated until the future is built.

Evidence: Epoch dependency direction is the primary failure mode in staged architecture systems. The CharacterOS Epoch model is only meaningful if each Epoch can be sealed, tested, and ratified independently.

Impact: During spec review, every dependency must be auditable as belonging to the current or an earlier Epoch. Forward references are a gate failure.

Revision Policy: Cannot be revised without replacing the Epoch model itself. Any revision must define a new inter-epoch relationship model and ratify it as a new Architectural Law.

Status: PROPOSED

LAW-004 — Character Identity Is Immutable Except Through Versioned Evolution

Once a character identity is ratified — its Invariant Kernel fields locked — those fields cannot be changed without creating a new versioned identity record. The prior version is archived, not deleted.

Rationale: Characters are IP assets. Their identity fields are the contract between FGE's creative system and any downstream use — rendering, narrative, licensing, derivative work. Silent mutation of a ratified identity corrupts the entire asset chain without a trace.

Evidence: The FGE Invariant Kernel / Open Evolution Space split was locked precisely because of this risk. The Anchor Method (L0 → L1 → L2) exists to enforce derive-never-improvise. This law makes that principle constitutional.

Impact: Every character identity field change requires a version increment and an archived prior state. Evolution is permitted; silent overwrite is not.

Revision Policy: Can be revised to redefine which fields belong to the Invariant Kernel vs. the Open Evolution Space, but cannot be revised to permit unversioned mutation of any ratified field.

Status: PROPOSED

LAW-005 — Implementations May Change; Contracts May Not Without Versioning

A contract, once ratified, is frozen at its version. Any change to a contract — including field additions, type changes, renamed keys, or altered invariants — requires a new contract version. The prior version remains valid until all dependent modules have migrated.

Rationale: Contracts are the stable interface layer. If contracts can change without versioning, every module built on them inherits an invisible, undatable breaking change. The entire governance system collapses into the same informal state it was designed to escape.

Evidence: This mirrors the Machine Translation Gate requirement: if a contract cannot be directly transformed into a schema or API spec, it is not yet a contract — it is a description. Versioned contracts are the only kind that can be machine-translated reliably.

Impact: Contract versioning discipline must be enforced from the first contract. No grandfather exceptions for early modules.

Revision Policy: Cannot be weakened to permit unversioned contract changes. May be

extended to add migration tooling requirements.

Status: PROPOSED

LAW-006 — Every Artifact Has Exactly One Owner

Every artifact in the CharacterOS system — law, register entry, contract, specification, module, implementation — must have exactly one named owner at all times. The owner is the sole authority for ratification decisions on that artifact. Ownership may be transferred, but the transfer must be logged.

Rationale: Shared ownership without a tiebreaker is no ownership. The Canon-Lock Authority model depends on clear accountability chains. An artifact with unclear ownership becomes de facto ungoverned.

Evidence: The FGE Canon-Lock Authority role was established precisely to resolve the governance ambiguity that emerges in multi-model workflows. This law extends that principle to artifact-level accountability.

Impact: Every artifact created in CharacterOS must have an owner field populated before it can be proposed. Ownerless artifacts are rejected at intake.

Revision Policy: Can be extended (e.g., to define deputy or delegated ownership patterns) but cannot be revised to permit shared ownership without a defined decision authority.

Status: PROPOSED

Law Summary Table

ID	Law	Status
LAW-001	Specifications precede implementations	PROPOSED
LAW-002	Modules communicate only through contracts	PROPOSED
LAW-003	Earlier epochs cannot depend on later epochs	PROPOSED
LAW-	Character identity is immutable except through versioned	

004	evolution	PROPOSED
LAW-005	Implementations may change; contracts may not without versioning	PROPOSED
LAW-006	Every artifact has exactly one owner	PROPOSED

Artifact 1 — Canonical Decisions Register

```

Artifact ID:      FGE-CDR-001
Version:          0.1.0
Status:           PROPOSED
Owner:            Keith Clemente
Last Ratified:    —
Depends On:       FGE-ARCH-LAWS-001
Required By:      FGE-CONTRACT-TEMPLATE-001, FGE-SEED-SPEC-001
Failure Mode:     Entries lack rationale, are undated, or reflect assumptions im-
                  posed from prior chat history without explicit ratification. A register
                  full of unexamined imports is not a register — it is a backlog.
Acceptance Gate: Every entry must be independently defensible by a cold reader with
                  prior FGE context. Entries that require background knowledge to
                  understand are incomplete.

```

Purpose

The Canonical Decisions Register (CDR) records the architectural decisions that are currently authoritative for CharacterOS. It is not a record of everything ever decided. It is the living set of decisions that govern the system today, with full rationale so any future decision-maker can understand not just what was decided, but why — and under what conditions a revision would be warranted.

Entries in the CDR are proposals until ratified by the Canon-Lock Authority. Ratified entries are frozen at their version. Changes produce new versioned entries; prior versions are archived.

Entry Template

Each CDR entry uses this structure:

```
CDR-[NUMBER]
Decision:      [What is decided, stated as a declarative rule]
Rationale:     [Why this decision was made – the argument, not the conclusion]
Evidence:      [What supports the rationale – prior work, observed failures, pr
Impact:        [What changes in the system as a result of this decision]
Revision Policy: [Under what conditions and by what process can this be changed]
Status:        PROPOSED | RATIFIED | DEPRECATED
Version:       [Semantic version of this entry]
Date:          [Date proposed or last revised]
```

Register Entries

CDR-001 — CharacterOS Atomic Unit Is the Break Pattern, Not the Character

Decision: The atomic unit of the CharacterOS civilization-physics model is the broken data pattern (damage class), not the character. Characters are derived artifacts — they emerge from break patterns, not the other way around.

Rationale: A character-first model anchors the system in presentation layer concerns (appearance, voice, narrative role) before the structural layer (what kind of damage does this identity encode?) is stable. If characters are atoms, the structural taxonomy becomes subordinate to creative decisions. If break patterns are atoms, characters become composable outputs with traceable structural origins.

Evidence: The FGE civilization-physics transition treated this as a founding-level event. The Break-Pattern Taxonomy v0.1 classifies eight damage classes as civilization preconditions. The existing CharacterOS genome schema was designed around invariant structural fields, not surface appearance fields.

Impact: All Epoch I and later module designs must treat break pattern identity as the upstream dependency of character identity. The Seed Specification must include break pattern classification as a required field.

Revision Policy: Can only be revised if a structural argument is made that a different atomic unit better satisfies the Machine Translation Gate and the cold-reader test simultaneously. Requires Canon-Lock Authority ratification.

Status: PROPOSED **Version:** 1.0.0 **Date:** 2026-06-26

CDR-002 — The Invariant Kernel Is the Contract Boundary for Character Identity

Decision: The five Invariant Kernel fields (sbid, material fingerprint, frequency cluster, core conflict, signal) are the contractual identity of a character. These fields may not be altered without creating a new versioned identity. All other fields belong to the Open Evolution Space and may change without versioning.

Rationale: A character IP asset requires a stable, auditable identity contract that survives rendering variation, narrative evolution, and platform migration. The Invariant Kernel defines "same character across all contexts." Without a hard contractual boundary, identity drift is invisible.

Evidence: The Invariant Kernel / Open Evolution Space split was locked in prior FGE governance work. The Anchor Method (L0 Master Anchor → L1 Render Prompt → L2 Scene Call) enforces derive-never-improvise, with the Master Anchor holding Invariant Kernel fields. The scar-suppression finding in The Professor's nine-render audit demonstrated that production environments can silently corrupt surface features — the Invariant Kernel exists precisely to flag when the core contract is violated.

Impact: The Seed Specification must expose all five Invariant Kernel fields as required, non-nullable, version-locked. Any module that reads or writes character identity must declare which field tier it touches.

Revision Policy: The list of Invariant Kernel fields may be revised through a formal canon ruling. Fields may migrate from Open Evolution Space to Invariant Kernel (promotion) or from Invariant Kernel to Open Evolution Space (demotion, requires strong justification). Each migration is a versioned CDR entry.

Status: PROPOSED **Version:** 1.0.0 **Date:** 2026-06-26

CDR-003 — Claude Is the Exclusive Engine for Gospel-Coherence Audit Work

Decision: All gospel-coherence audits — validation of character identity against canon, contract compliance review, specification gate evaluation — are performed by Claude at temperature 0.2. Grok is the generative layer. Claude is the constitutional audit layer. These roles do not interchange.

Rationale: Multi-model workflows fail when model roles are ambiguous. Grok's generative strength is its ability to produce high-volume, stylistically consistent output. Claude's strength is structured reasoning, contract-checking, and identifying drift. Mixing these roles introduces inconsistency that is invisible at the output level but

corrosive at the governance level.

Evidence: The Prospect Freshness Audit Protocol formally established Claude as the exclusive audit engine. The FGE DIS (Decision Intelligence System) architecture routes governance decisions through Claude. This CDR entry makes that role assignment constitutional rather than operational.

Impact: Any system or workflow that routes canon audit decisions through a non-Claude model is non-compliant with CharacterOS governance. This applies to automated pipelines as well as manual review.

Revision Policy: May be revised if a model demonstrably outperforms Claude on the canonical audit task set. Revision requires a structured comparison test, not preference. The test protocol must itself be ratified before the comparison is run.

Status: PROPOSED **Version:** 1.0.0 **Date:** 2026-06-26

CDR-004 — Epoch Boundaries Are Hard Gates, Not Milestones

Decision: An Epoch boundary is a hard gate: work on Epoch N+1 may not begin until all Epoch N deliverables have passed their acceptance gates and been ratified by the Canon-Lock Authority. Partial Epoch completion does not permit partial advancement.

Rationale: Milestones permit forward work to begin while prior work is still open. In a specification-governed system, this creates downstream artifacts that depend on unratified upstream decisions. When the upstream decision changes during ratification, the downstream artifacts inherit the instability. Hard gates eliminate this class of failure entirely.

Evidence: The DIS architecture critique identified “layers buildable against existing tool stack” vs. “layers deferred pending backend” — this is the gate model in practice. The epoch structure was designed specifically to prevent the “sprawling all-at-once system” failure mode.

Impact: Epoch I work begins only after Epoch 0’s four artifacts are ratified. This is not negotiable. Time pressure does not override the gate.

Revision Policy: Cannot be revised to permit partial advancement. May be revised to define a formal exception process for isolated, non-dependent exploratory work, provided that work carries a DRAFT status and cannot be cited as ratified until the gate is passed.

Status: PROPOSED **Version:** 1.0.0 **Date:** 2026-06-26

CDR-005 — Canon Authority Flows From Ruling, Not Completeness

Decision: A decision is canonical because the Canon-Lock Authority has ratified it, not because it is thorough, well-argued, or widely accepted. Incomplete ratifications do not become canonical through consensus or accumulated usage.

Rationale: Systems that treat completeness or community acceptance as canonicity signals drift toward governance by accumulation — informal decisions that gradually acquire weight without ever being formally ratified. The FGE architecture is built on the principle that the canon is what the canon-lock says it is, full stop.

Evidence: This was stated as a founding invariant in the FGE civilization-physics transition: “canon authority flows from ruling not completeness.” This CDR entry ratifies that invariant into the governance layer.

Impact: Every artifact review session must end with either a ratification ruling, a list of open items required before ratification, or an explicit deferral with a reason. There is no implicit ratification.

Revision Policy: Cannot be revised to introduce consensus-based or completeness-based canonicity paths. May be extended to define delegation patterns (e.g., the Canon-Lock Authority explicitly delegates ratification authority for a defined scope).

Status: PROPOSED **Version:** 1.0.0 **Date:** 2026-06-26

Artifact 2 — Universal Module Contract Template

Artifact ID:	FGE-CONTRACT-TEMPLATE-001
Version:	0.1.0
Status:	PROPOSED
Owner:	Keith Clemente
Last Ratified:	—
Depends On:	FGE-ARCH-LAWS-001, FGE-CDR-001
Required By:	FGE-SEED-SPEC-001 and all future module specifications
Failure Mode:	Fields are present but ambiguous — they can be satisfied by multiple contradictory implementations. A template that does not force precise is a checklist, not a contract.
Acceptance Gate:	(1) Cold-reader test: a developer with no FGE context can implement a compliant module using only this template and the completed specifications (2) Machine Translation Gate: every field in this template can be directly mapped to a JSON schema field, API parameter, or database column without introducing information not present in the template

Purpose

The Universal Module Contract Template (UMCT) is the language every CharacterOS module speaks. Every specification written for CharacterOS must be structured using this template. If a specification cannot be expressed in this template without leaving information outside it, the template has a defect and must be revised before the specification is accepted.

The template is the proof layer. When the first real specification (the Character Seed Specification) is written using it, any template fields that are ambiguous, redundant, or missing will become visible. That is by design.

Artifact Metadata Block

Every specification begins with this block, fully populated:

```
Artifact ID:      [FGE-{DOMAIN}-{MODULE}-{NUMBER}]
Version:          [Semantic version: MAJOR.MINOR.PATCH]
Status:           PROPOSED | RATIFIED | DEPRECATED
Owner:            [Named individual – not a team, not a role]
Last Ratified:    [ISO 8601 date, or – if not yet ratified]
Depends On:       [Artifact IDs this spec requires to exist and be ratified]
Required By:      [Artifact IDs that cannot exist without this spec]
Failure Mode:     [One to three sentences describing what this artifact looks like
                  when it is wrong or incomplete]
Acceptance Gate:  [The specific, testable conditions that must be true for this
                  artifact to pass from PROPOSED to RATIFIED]
```

Rules:

- No field may be left blank. Use – only for `Last Ratified` when not yet ratified.
- `Depends On` and `Required By` must reference Artifact IDs, not descriptions.
- `Failure Mode` must describe observable symptoms, not internal states.
- `Acceptance Gate` must be binary-testable. "Good enough" is not a gate.

Module Specification Body

After the metadata block, every module specification contains the following sections in this order:

1. Purpose

Format: Two to five sentences.

Required content:

- What problem this module solves
- What the system state looks like before this module runs
- What the system state looks like after this module runs

Forbidden content:

- Implementation details
 - References to specific tools, languages, or frameworks
 - Aspirational language ("will eventually," "may support")
-

2. Inputs

For each input, provide:

Input ID:	[Unique identifier within this spec, e.g., IN-001]
Name:	[Human-readable field name]
Type:	[Primitive or reference type – string, integer, ISO 8601 date, Artifact ID reference, enum{A B C}, etc.]
Required:	[YES NO]
Source:	[Which module or artifact produces this input]
Invariant:	[A constraint that must be true about this value for the module to accept it. If none, state NONE explicitly.]
Failure:	[What happens if this input is absent or violates its invariant]

Rules:

- Every input that is marked Required:YES must have a non-NONE Invariant or a documented reason why no invariant is possible.
- Inputs sourced from other modules must reference the source module's Artifact ID.
- Inputs sourced from external systems must describe the interface, not the system.

3. Outputs

For each output, provide:

Output ID:	[Unique identifier within this spec, e.g., OUT-001]
Name:	[Human-readable field name]
Type:	[Same type vocabulary as Inputs]
Guaranteed:	[YES CONDITIONAL NO]
Consumer:	[Which module or artifact consumes this output]
Invariant:	[A constraint that must be true about this value when it is emitted]
Failure:	[What the module emits when output cannot be produced – null, error object, partial record, etc. Must be explicit. NOTHING is not valid]

Rules:

- Every Guaranteed:YES output must have a non-NONE Invariant.
- Every CONDITIONAL output must specify the condition under which it is emitted.
- Failure behavior for every output must be explicit. A module that silently produces nothing on failure violates LAW-002.

4. Invariants

Module-level invariants: conditions that must be true at all times while the module is operating, regardless of specific input or output values.

Invariant ID:	[INV-001, INV-002, etc.]
Statement:	[A declarative assertion about system state]
Violation:	[What the module must do if this invariant is detected as violated halt, log, emit error contract, escalate]

Rules:

- Minimum one invariant per module.
- Invariants must be falsifiable — there must be a condition under which they could be detected as violated.
- “The module will work correctly” is not an invariant.

5. Failure Conditions

Enumerate the known ways this module can fail, beyond individual input/output failures:

Failure ID: [FAIL-001, FAIL-002, etc.]
Condition: [What state causes this failure]
Detection: [How the failure is detectable – not how it is handled]
Response: [What the module emits or does when this failure is detected]
Recovery: [Whether this failure is recoverable, and if so, how]

Rules:

- At minimum, define failures for: (a) upstream contract violation, (b) internal processing error, (c) downstream consumer unavailable.
 - Recovery must be explicit. "TBD" is not a valid entry.
-

6. Acceptance Tests

Enumerate the tests that, if all pass, constitute evidence that the module behaves according to this specification:

Test ID: [TEST-001, TEST-002, etc.]
Description: [What is being tested, in plain language]
Input State: [The exact input conditions for this test]
Expected Output: [The exact expected output, referencing Output IDs]
Pass Criterion: [Binary: what does pass look like vs. fail]
Gate: [Which Acceptance Gate does this test satisfy?]

Rules:

- Each Acceptance Gate must be covered by at least one test.
 - Tests must be runnable by the cold reader — they must not require internal system knowledge.
 - Tests must reference Output IDs, not describe output in prose.
-

7. Version History

Version: [Semantic version]
Date: [ISO 8601]
Author: [Named individual]
Changes: [What changed from the prior version – use explicit field-level 1

Breaking: [YES | NO – does this version break compatibility with prior cons
Migration: [If Breaking:YES, what is the migration path?]

Rules:

- Version 0.1.0 is always the first PROPOSED entry.
 - RATIFIED status requires a version history entry with the ratification date.
 - Breaking changes require the prior version to remain documented and available.
-

Template Completeness Self-Test

Before submitting any specification for ratification, the author must confirm:

- All metadata block fields are populated (no blanks except permitted –)
 - All Input invariants are either non-NONE or have documented justification
 - All Output failure behaviors are explicit (no silent failures)
 - All module invariants are falsifiable
 - All three minimum failure conditions are addressed (upstream, internal, downstream)
 - All Acceptance Gates are covered by at least one test
 - Machine Translation Gate: every field maps to a JSON schema field without adding information
 - Cold-Reader Gate: specification has been reviewed by someone with no prior FGE context, OR owner has performed a cold-read simulation and documented any knowledge gaps found and resolved
-
-

Artifact 3 — Character Seed Specification v1.0

Artifact ID: FGE-SEED-SPEC-001
Version: 0.1.0
Status: PROPOSED
Owner: Keith Clemente
Last Ratified: —

Depends On: FGE-ARCH-LAWS-001, FGE-CDR-001, FGE-CONTRACT-TEMPLATE-001
Required By: FGE-MODULE-SEED-INTAKE-001 (Epoch I), all character-producing mo
Failure Mode: The spec appears complete but relies on implicit knowledge of FG conventions, prior character work, or undocumented break-pattern to interpret. A cold reader cannot generate a valid seed without
Acceptance Gate: (1) Cold-reader test: a developer with no FGE context reads this and produces a valid seed record with zero clarifying questions.
(2) Machine Translation Gate: this spec can be transformed into JSON schema with no fields left to implementer interpretation.
(3) Template Completeness Self-Test passes (all checkboxes clear

1. Purpose

The Character Seed Specification defines the minimal, sufficient data structure required to uniquely identify and initiate the genesis of a CharacterOS character. A seed is the input to the character generation pipeline — it contains exactly the information required to begin, and nothing more. A valid seed does not describe a complete character. It describes a complete starting position: the break pattern origin, the invariant identity anchors, and the structural coordinates from which all downstream generation is derived.

Before this module runs: no character record exists. There is an intent to create a character and a set of input values that have not been validated or structured.

After this module runs: a validated, versioned seed record exists. It is stored in the character registry with a unique identifier, a SHA-256 fingerprint, and a full audit trail. It is ready to be consumed by the Seed Intake Module (Epoch I).

2. Inputs

IN-001

Input ID: IN-001
Name: character_name
Type: string
Required: YES
Source: Human operator (Canon-Lock Authority or delegated creator)
Invariant: Length 2-64 characters. UTF-8. No leading/trailing whitespace.
Must not duplicate an existing ratified character name in the reg
Failure: If absent: seed record rejected at intake with error SEED-ERR-001
If duplicate: seed record rejected with error SEED-ERR-002.

IN-002

Input ID: IN-002
Name: sbid
Type: string – format: SB{integer} where integer is 3+ digits, e.g. SB3
Required: YES
Source: Human operator
Invariant: Must conform to SB{NNN+} format. Must not duplicate an existing s
in the registry.
Failure: If absent or malformed: SEED-ERR-003.
If duplicate: SEED-ERR-004.

IN-003

Input ID: IN-003
Name: material_fingerprint
Type: string – enum from Material Fingerprint Registry
Current valid values: Obsidian | Kintsugi | Pearl-Nacre |
Mystic-Pearl | Ember | Ash | Void | Feral-Gloss
Required: YES
Source: Human operator
Invariant: Value must exist in the Material Fingerprint Registry at time of
creation. Registry is versioned; seed records the registry versio
Failure: If absent: SEED-ERR-005.
If value not in registry: SEED-ERR-006 with registry version logg

IN-004

Input ID: IN-004
Name: frequency_cluster
Type: string – enum{Mirror-Siren | Consumption-Succubus | Control-Vampi
Required: YES
Source: Human operator
Invariant: Must be one of the three defined Orb class values. No custom valu
permitted in Epoch 0 or Epoch I.
Failure: If absent: SEED-ERR-007.
If value outside enum: SEED-ERR-008.

IN-005

Input ID: IN-005
Name: core_conflict
Type: string
Required: YES

Source: Human operator

Invariant: Length 10-280 characters. Must be a declarative statement of int
structural tension – not a backstory summary, not a trait list.
Example of valid: "Identity constructed entirely from other peopl
signals cannot survive contact with someone who refuses to reflec
Example of invalid: "She is mysterious and dangerous."

Failure: If absent: SEED-ERR-009.
If length out of bounds: SEED-ERR-010.
If validation audit flags non-declarative form: SEED-WARN-001
(warning, not rejection – requires human review flag on record).

IN-006

Input ID: IN-006

Name: signal

Type: string

Required: YES

Source: Human operator

Invariant: Length 4-80 characters. A single phrase or short sentence describ
the dominant perceptual or energetic output of this character.
Example of valid: "Gravity without warmth."
Example of invalid: "She radiates dark energy and makes people fe
drawn to her but also unsettled." (too long, too descriptive)

Failure: If absent: SEED-ERR-011.
If length out of bounds: SEED-ERR-012.

IN-007

Input ID: IN-007

Name: break_pattern_class

Type: string – enum from Break-Pattern Taxonomy
Current valid values: Structural-Fracture | Identity-Erasure |
Signal-Corruption | Boundary-Collapse | Temporal-Displacement |
Consumption-Loop | Control-Inversion | Void-Formation

Required: YES

Source: Human operator

Invariant: Must be one of the eight damage classes from Break-Pattern Taxono
Records the taxonomy version at time of seed creation.

Failure: If absent: SEED-ERR-013.
If value not in taxonomy: SEED-ERR-014 with taxonomy version logg

IN-008

Input ID: IN-008

Name: origin_epoch

Type:	integer
Required:	YES
Source:	System (auto-populated)
Invariant:	Must equal the current ratified Epoch number at time of seed creation. Cannot be set by human operator. Cannot be retroactively changed.
Failure:	If system cannot determine current epoch: SEED-ERR-015. Halt.

IN-009

Input ID:	IN-009
Name:	creator_id
Type:	string
Required:	YES
Source:	System (authenticated session)
Invariant:	Must resolve to a valid, authenticated operator identity in the CharacterOS operator registry.
Failure:	If absent or unresolvable: SEED-ERR-016. Halt.

IN-010

Input ID:	IN-010
Name:	notes
Type:	string
Required:	NO
Source:	Human operator
Invariant:	If present: length 0-1000 characters. Free text. Not validated. Explicitly not part of the Invariant Kernel. Not included in SHA-256 fingerprint calculation.
Failure:	If present and exceeds length: truncated with SEED-WARN-002 logged

3. Outputs

OUT-001

Output ID:	OUT-001
Name:	seed_record
Type:	object — see Seed Record Schema below
Guaranteed:	CONDITIONAL — emitted only when all required inputs pass validation
Consumer:	Character Registry (storage), Seed Intake Module (Epoch I, FGE-MO)
Invariant:	seed_record.fingerprint is the SHA-256 hash of the concatenated Invariant Kernel fields in canonical order: SHA-256(sbid + " " + material_fingerprint + " " + frequency_clust + " " + core_conflict + " " + signal)

seed_record.version is "1.0.0" for all records created against th
seed_record.status is "PROPOSED" at creation. Never "RATIFIED" at
Failure: If validation fails: OUT-001 is not emitted. OUT-002 is emitted i

OUT-002

Output ID: OUT-002
Name: validation_error_report
Type: object – {error_codes: string[], warnings: string[], input_snapsh
Guaranteed: CONDITIONAL – emitted only when validation produces errors
Consumer: Human operator (returned to submitter)
Invariant: error_codes array contains at least one entry when emitted.
input_snapshot contains the exact input values submitted, unmodif
Failure: If OUT-002 cannot be constructed (system error): SEED-ERR-099.
Log system error. Return generic failure indicator to operator.

Seed Record Schema

```
{
  "seed_id": "string – UUID v4, system-generated",
  "version": "string – semver, e.g. '1.0.0'",
  "status": "string – enum{PROPOSED|RATIFIED|DEPRECATED}",
  "created_at": "string – ISO 8601 datetime with timezone",
  "origin_epoch": "integer",
  "creator_id": "string",
  "spec_version": "string – Artifact ID + version: 'FGE-SEED-SPEC-001@1.0.'

  "invariant_kernel": {
    "sbid": "string",
    "material_fingerprint": "string",
    "frequency_cluster": "string",
    "core_conflict": "string",
    "signal": "string"
  },

  "identity": {
    "character_name": "string",
    "break_pattern_class": "string",
    "break_pattern_taxonomy_version": "string"
  },

  "fingerprint": "string – SHA-256 hex digest of invariant_kernel fie
  "material_registry_version": "string",
```

```

"open_evolution_space": {
  "notes": "string | null"
},

"audit": {
  "validation_warnings": "string[]",
  "ratification_log": "object[] - append-only, one entry per status change"
}
}

```

4. Invariants

INV-001

Invariant ID: INV-001

Statement: The fingerprint field is always the SHA-256 hash of the five Invariant Kernel fields in canonical order. It is never computed from any other fields. It is never manually set.

Violation: If a stored seed_record's fingerprint does not match the recomputed hash of its invariant_kernel fields: flag record as CORRUPTED, halt processing, emit SEED-SYS-ERR-001, escalate to Canon-Lock Authority.

INV-002

Invariant ID: INV-002

Statement: The invariant_kernel fields of a RATIFIED seed record are immutable. No system or operator action may alter them without first creating a new versioned seed record and deprecating the prior one.

Violation: If a write operation attempts to alter invariant_kernel fields on a RATIFIED record: reject operation with SEED-SYS-ERR-002. Log attempt with operator identity and timestamp.

INV-003

Invariant ID: INV-003

Statement: A seed record's status may only advance in one direction: PROPOSED → RATIFIED, or PROPOSED → DEPRECATED, or RATIFIED → DEPRECATED. Status cannot move backward.

Violation: If a status transition is attempted that does not follow this graph, reject operation with SEED-SYS-ERR-003.

5. Failure Conditions

FAIL-001 — Upstream Contract Violation

Failure ID: FAIL-001

Condition: One or more required inputs are absent, malformed, or violate the stated invariants.

Detection: Validation pass at intake. Each violated input produces a specific error code (SEED-ERR-001 through SEED-ERR-016).

Response: Emit OUT-002 (validation_error_report). Do not emit OUT-001. Do not create a partial seed record.

Recovery: Recoverable. Operator corrects inputs and resubmits.

FAIL-002 — Internal Processing Error

Failure ID: FAIL-002

Condition: Fingerprint computation fails, UUID generation fails, or registry lookup produces an unexpected result.

Detection: System exception during processing. Distinguished from FAIL-001 by error code range SEED-SYS-ERR-*.

Response: Emit SEED-SYS-ERR-099 with diagnostic payload. Do not emit OUT-001 or OUT-002. Log full input_snapshot for recovery.

Recovery: Recoverable with system intervention. Input snapshot preserved for resubmission once system issue is resolved.

FAIL-003 — Downstream Consumer Unavailable

Failure ID: FAIL-003

Condition: Character Registry is unavailable at time of seed record write.

Detection: Write operation to registry returns connection/timeout error.

Response: Seed record is computed and validated successfully but held in a PENDING_WRITE state. Emit SEED-WARN-003 to operator. Do not report success until registry write is confirmed.

Recovery: Recoverable. Retry with exponential backoff. If registry remains unavailable after retry limit: escalate to operator with full seed_record payload for manual insertion.

6. Acceptance Tests

TEST-001 — Valid seed record is produced from complete, valid inputs

Test ID: TEST-001

Description: Confirm that submitting all required inputs with valid values produces a well-formed seed record with correct fingerprint.

Input State: IN-001: "Raven Voss" | IN-002: "SB330" | IN-003: "Obsidian" | IN-004: "Control-Vampire" | IN-005: "Power accumulated through other people's collapse cannot survive the moment someone refuses to fall." | IN-006: "Dominion without warmth." | IN-007: "Control-Inversion" | IN-008: 0 | IN-009: "operator-001"

Expected Output: OUT-001 emitted. seed_record.status = "PROPOSED".
seed_record.fingerprint = SHA-256("SB330|Obsidian|Control-Vampir Power accumulated...|Dominion without warmth.") – exact hash.
OUT-002 not emitted.

Pass Criterion: OUT-001 is present. Fingerprint matches independent computation.
No validation errors present.

Gate: Cold-Reader Gate, Machine Translation Gate

TEST-002 — Duplicate sbid is rejected

Test ID: TEST-002

Description: Confirm that submitting an sbid already present in the registry produces a validation error and no seed record.

Input State: All valid inputs except IN-002: "SB330" where SB330 already exist in registry as a RATIFIED record.

Expected Output: OUT-002 emitted with error_codes containing "SEED-ERR-004".
OUT-001 not emitted.

Pass Criterion: OUT-001 absent. OUT-002.error_codes includes SEED-ERR-004.

Gate: Cold-Reader Gate

TEST-003 — Invariant Kernel fields are fingerprinted; notes field is not

Test ID: TEST-003

Description: Confirm that changing the notes field produces the same fingerprint and changing any Invariant Kernel field produces a different fingerprint.

Input State: Two submissions identical except IN-010 (notes) differs.

Expected Output: Both OUT-001 records have identical fingerprint values.

Pass Criterion: Fingerprints match between the two records.

Gate: Machine Translation Gate, INV-001 verification

TEST-004 — Missing required field produces specific error code

Test ID: TEST-004

Description: Confirm that each required input, when absent, produces its specific error code and not a generic failure.

Input State: Seven separate submissions, each omitting exactly one required field (IN-001 through IN-007 individually).

Expected Output: Seven OUT-002 reports, each containing the specific SEED-ERR-*

code corresponding to the omitted field.

Pass Criterion: Each error report contains exactly the expected error code.

No OUT-001 emitted in any of the seven runs.

Gate: Cold-Reader Gate

TEST-005 — Machine Translation Gate proof

Test ID: TEST-005

Description: Confirm that the Seed Record Schema can be expressed as a complete JSON Schema (draft-07 or later) with no fields requiring interpretive commentary.

Input State: The Seed Record Schema as defined in Section 3 of this specification

Expected Output: A valid JSON Schema document that validates all fields, types, enums, and format constraints defined in this spec.

Pass Criterion: The generated JSON Schema, when used to validate TEST-001's OUT-001 output, passes without error. No schema field required additional comments to implement.

Gate: Machine Translation Gate

7. Version History

Version: 0.1.0

Date: 2026-06-26

Author: Keith Clemente (via Claude, constitutional audit layer)

Changes: Initial draft. First application of FGE-CONTRACT-TEMPLATE-001.

Breaking: N/A — first version

Migration: N/A

Epoch 0 Ratification Checklist

This checklist must be completed by the Canon-Lock Authority before Epoch I work begins.

Artifact 0 — Architectural Laws

- All six laws reviewed
- Each law passes: "Could a developer with no FGE context apply this unambiguously?"

- No law is too abstract to decide a real case
- Status changed to RATIFIED with date

Artifact 1 — Canonical Decisions Register

- All five entries reviewed
- Each entry has rationale, evidence, impact, and revision policy
- No entry requires background knowledge to interpret
- Status changed to RATIFIED with date

Artifact 2 — Universal Module Contract Template

- Template completeness self-test passes (all checkboxes cleared)
- Machine Translation Gate: every field maps to a JSON schema field
- Cold-reader test passed or simulated
- Status changed to RATIFIED with date

Artifact 3 — Character Seed Specification v1.0

- All three acceptance gates cleared: Cold-Reader, Machine Translation, Template Self-Test
- TEST-005 (Machine Translation Gate proof) executed and passed
- Status changed to RATIFIED with date

Epoch Gate

- All four artifacts at Status: RATIFIED
 - Gate question answered YES: "Can the Seed Spec be fully expressed using the Contract Template?"
 - Gate question answered YES: "Can another developer implement the Seed Specification without asking the original author any questions?" — tested against a cold reader with no FGE context
 - Gate question answered YES: "Can this artifact be transformed into JSON, a database schema, API contracts, or executable code without introducing information not already present?"
 - Canon-Lock Authority signs off: Epoch 0 is closed. Epoch I may begin.
-

